

SIMATS ENGINEERING

ASSESSMENT TOOL 2

**COURSE NAME: CRYPTOGRAPHY AND NETWORK
SECURITY COURSE CODE: CSA51**

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security.
(BL4,BL5)

Scenario-Based : 20%

QUESTIONS:

Total Marks: 50

Annexure A

Code of Conduct:

I _____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario-based

1. Secure Data Transmission using Block Cipher

A company is transmitting repeated financial data blocks over a network. Initially, they used ECB mode and noticed patterns in ciphertext.

- A. Explain why this issue occurs.
- B. Suggest a more secure block cipher mode and justify your choice based on the scenario.

2. Choosing an Encryption Algorithm for Banking System

A banking application needs high security and moderate performance for encrypting customer transactions. The available options are DES, 3-DES, and Blowfish.

- A. Identify the most suitable algorithm for this scenario.
- B. Justify your choice based on security strength and efficiency.

3. Key Exchange in an Unsecured Network

Two users want to establish a secure communication channel over a public network using Diffie-Hellman key exchange. However, the network is vulnerable to attacks.

- A. Explain how the key exchange works in this scenario.
- B. Identify a possible security threat and suggest a solution to mitigate it.

4. Public Key Cryptography for Email Security

An organization plans to implement RSA for secure email communication.

- A. Describe how RSA can be used for encryption and decryption in this scenario.
- B. Explain how key distribution and management play a role in maintaining security.

5. Cryptography for IoT Devices

A smart home system uses low-power IoT devices that need secure communication. The designer must choose between RSA and Elliptic Curve Cryptography (ECC).

- A. Analyze the constraints of IoT devices.
- B. Recommend a suitable cryptographic approach and justify your answer.

RUBRICS FOR CALCULATION:

Criteria	Weight a g e (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions	Makes reasonable decisions with some	Makes weak decisions with limited	No clear decisions made

g		with strong rationale	justification	justification	or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Question 1. Secure Data Transmission using Block Cipher

Scenario

A company is transmitting repeated financial data blocks over a network. Initially, they used Electronic Codebook (ECB) mode for encryption. After some time, they noticed that patterns in the encrypted data (ciphertext) were still visible. The company now wants to understand why this happened and choose a more secure block cipher mode to protect confidential financial information.

A. Explain why this issue occurs.

Electronic Codebook (ECB) is one of the simplest modes of operation for a block cipher. In ECB mode, the plaintext is divided into fixed-size blocks, and each block is encrypted independently using the same secret key. Since every block is treated separately, if two plaintext blocks contain identical data, they will always produce the same ciphertext block.

In a financial system, many records contain repeated information such as customer IDs, account numbers, transaction headers, account types, or fixed message formats. When these repeated data blocks are encrypted using ECB, the encrypted output also repeats. As a result, visible patterns remain in the ciphertext.

Although an attacker may not immediately know the actual content of the encrypted message, the repeated ciphertext blocks reveal the structure of the original data. By observing these patterns, an attacker can determine where information is repeated, compare multiple encrypted messages, and perform statistical or pattern analysis attacks. This weakens confidentiality and may help an attacker gather useful information about the organization's financial transactions.

For example, if the plaintext blocks are:

Transaction A: Salary Payment

Transaction B: Salary Payment

Both transactions contain identical data. ECB encrypts them independently using the same key, producing identical ciphertext blocks.

Plaintext:

Salary Payment → Ciphertext: X7A91F

Salary Payment → Ciphertext: X7A91F

Since both ciphertext blocks are identical, anyone monitoring the communication can recognize that the same information has been transmitted multiple times.

Therefore, ECB mode is not suitable for applications that process highly sensitive information such as banking, finance, healthcare, military communication, or government records because it leaks patterns even though the actual data remains encrypted.

Problems with ECB Mode

ECB mode has several security weaknesses:

It encrypts each block independently.

Identical plaintext blocks produce identical ciphertext blocks.

Patterns in the original data remain visible.

It is vulnerable to pattern analysis attacks.

It does not hide the structure of the original message.

It is not recommended for encrypting confidential financial information.

B. Suggest a more secure block cipher mode and justify your choice based on the scenario.

A much better choice for this scenario is **Cipher Block Chaining (CBC) mode**.

In CBC mode, before encrypting each plaintext block, it is combined (using the XOR operation) with the previous ciphertext block. The first block uses a randomly generated **Initialization Vector (IV)**. Because of this chaining process, even if two plaintext blocks are identical, their ciphertext will be completely different.

The encryption process is as follows:

First plaintext block $\rightarrow$ XOR with Initialization Vector (IV) $\rightarrow$ Encryption $\rightarrow$ Ciphertext Block 1

Second plaintext block $\rightarrow$ XOR with Ciphertext Block 1 $\rightarrow$ Encryption $\rightarrow$ Ciphertext Block 2

Third plaintext block $\rightarrow$ XOR with Ciphertext Block 2 $\rightarrow$ Encryption $\rightarrow$ Ciphertext Block 3

Since every ciphertext block depends on the previous ciphertext, repeated plaintext blocks no longer produce identical encrypted outputs.

For example, consider two identical plaintext blocks:

Plaintext:

Salary Payment

Salary Payment

CBC Encryption Output:

Ciphertext 1: 8DF3A91

Ciphertext 2: C74E5B2

Although the plaintext is exactly the same, the ciphertext is completely different because the second block depends on the first encrypted block.

This property makes CBC much more secure than ECB because attackers cannot identify repeated information by observing the ciphertext.

Advantages of CBC Mode

CBC eliminates visible patterns in encrypted data.

Identical plaintext blocks generate different ciphertext blocks.

The Initialization Vector (IV) adds randomness to every encryption process.

It provides stronger confidentiality than ECB.

It is widely used for protecting sensitive financial and business data.

It significantly reduces the possibility of pattern analysis attacks.

Comparison Between ECB and CBC

Feature	ECB Mode	CBC Mode
Encryption Method	Each block encrypted independently	Each block depends on previous ciphertext
Identical Plaintext	Produces identical ciphertext	Produces different ciphertext
Pattern Visibility	Yes	No
Initialization Vector	Not used	Required
Security Level	Low	High

Feature	ECB Mode	CBC Mode
No	Yes	

Justification

Since the company is transmitting repeated financial records, protecting customer information and transaction details is essential. ECB exposes repeated patterns that attackers can analyze, making it unsuitable for secure communication. CBC hides these patterns by introducing dependency between ciphertext blocks and using a random Initialization Vector.

Therefore, **CBC mode is the most suitable choice** because it provides stronger confidentiality, prevents pattern leakage, and offers much better protection for financial transactions transmitted over a network.

Conclusion

The problem occurs because ECB encrypts identical plaintext blocks into identical ciphertext blocks, exposing patterns that attackers can observe. This weakness makes ECB unsuitable for transmitting confidential financial information. CBC mode solves this problem by using an Initialization Vector and chaining each block with the previous ciphertext, ensuring that identical plaintext blocks generate different ciphertext. As a result, CBC provides higher security, better confidentiality, and is a more appropriate choice for secure financial data transmission.

Question 2. Choosing an Encryption Algorithm for Banking System

Scenario

A banking application needs an encryption algorithm that provides **high security** while maintaining **moderate performance** for protecting customer transactions. The available encryption algorithms are **DES, 3-DES, and**

Blowfish. The objective is to identify the most suitable algorithm and justify the selection based on security and efficiency.

A. Identify the most suitable algorithm for this scenario.

Among the three available algorithms, **Blowfish** is the most suitable choice for the banking application.

DES is no longer considered secure because its key size is too small, making it vulnerable to brute-force attacks. Although 3-DES improves security by applying the DES algorithm three times, it requires more computation and is slower. Blowfish provides a better balance between security and performance by offering a larger key size, faster encryption, and efficient processing of large amounts of financial data.

Since banking systems require both strong protection and good processing speed, Blowfish is the most appropriate algorithm among the given options.

B. Justify your choice based on security strength and efficiency.

Banking applications handle highly confidential information such as customer account numbers, passwords, ATM PINs, credit card details, transaction records, and online payment information. Therefore, the encryption algorithm must ensure confidentiality, integrity, and resistance against attacks while also processing transactions quickly.

Analysis of DES

Data Encryption Standard (DES) is one of the earliest symmetric encryption algorithms.

DES uses:

Block Size = 64 bits

Key Size = 56 bits

Although DES was once widely used, advances in computing technology have made it insecure. A 56-bit key is too small, allowing attackers to perform brute-force attacks by trying every possible key until the correct one is found.

Advantages of DES

DES is simple to implement.

Encryption and decryption are relatively fast.

It requires less memory.

Disadvantages of DES

Very small 56-bit key.

Easily broken using modern computers.

Not suitable for banking or financial applications.

Provides low security against current cyber threats.

Therefore, DES should not be selected for modern banking systems.

Analysis of 3-DES

Triple DES (3-DES) was developed to improve the security of DES.

Instead of encrypting the data once, it encrypts the data three times using different keys.

Encryption Process:

Plaintext

↓

DES Encryption

↓

DES Decryption

↓

DES Encryption

↓

Ciphertext

Because of this triple encryption process, 3-DES provides much stronger security than DES.

Advantages of 3-DES

Much stronger security than DES.

Difficult to break using brute-force attacks.

Well-tested and reliable.

Suitable for protecting sensitive information.

Disadvantages of 3-DES

Three encryption operations increase execution time.

Consumes more CPU resources.

Slower than modern encryption algorithms.

Not efficient for systems processing thousands of banking transactions every second.

Although 3-DES offers good security, its lower performance makes it less suitable for modern banking applications.

Analysis of Blowfish

Blowfish is a symmetric block cipher designed to provide both strong security and high performance.

Features of Blowfish:

Block Size = 64 bits

Key Size = Variable (32 to 448 bits)

Variable key lengths allow organizations to choose stronger encryption based on their security requirements.

Unlike DES, Blowfish supports much larger keys, making brute-force attacks practically impossible.

Blowfish is also designed for high-speed software implementation and requires fewer computational resources than 3-DES.

Advantages of Blowfish

- Supports key sizes up to 448 bits.
- Much stronger protection against brute-force attacks.
- Fast encryption and decryption.
- Efficient for processing large amounts of banking transactions.
- Requires less computational overhead than 3-DES.
- Widely trusted for protecting confidential information.
- Good balance between security and performance.

Disadvantages of Blowfish

- Uses a 64-bit block size, which is smaller than some newer encryption algorithms.
- Key generation is relatively slow, although encryption itself is very fast.

- Despite these limitations, Blowfish remains a strong and practical encryption algorithm for many applications.

Comparison of DES, 3-DES and Blowfish

Feature	DES	3-DES	Blowfish
Key Size	56 bits	112 or 168 bits	32–448 bits
Block Size	64 bits	64 bits	64 bits
Security	Low	High	Very High
Speed	Fast	Slow	Fast
Brute Force Resistance	Poor	Good	Excellent
CPU Usage	Low	High	Moderate
Suitable for Banking	No	Acceptable	Yes

Justification

The banking application requires both **high security** and **moderate performance**. DES cannot provide sufficient security because of its small key size. Although 3-DES improves security, it performs three encryption operations, making it slower and less efficient for handling a large number of customer transactions.

Blowfish provides strong encryption with a variable key size of up to 448 bits, making it highly resistant to brute-force attacks. It also performs encryption much faster than 3-DES while maintaining a high level of security. These features make Blowfish well suited for banking systems that require secure and efficient transaction processing.

Conclusion

Among the available algorithms, **Blowfish is the most suitable choice** for the banking application. DES is outdated and vulnerable due to its small key size, while 3-DES offers better security but suffers from slower performance. Blowfish provides an excellent balance of **strong security, fast encryption, low computational overhead, and efficient processing**, making it the best option for protecting customer transactions in a banking environment.

Question 3. Key Exchange in an Unsecured Network

Scenario

Two users want to establish a secure communication channel over a public (unsecured) network using the **Diffie–Hellman Key Exchange** algorithm. Since the communication takes place over the Internet, attackers may intercept the exchanged messages. The objective is to understand how Diffie–Hellman establishes a shared secret key and identify the possible security threats along with suitable countermeasures.

A. Explain how the key exchange works in this scenario.

Diffie–Hellman Key Exchange is a public-key cryptographic protocol that allows two users to establish a common secret key over an insecure network without directly transmitting the secret key. The shared secret key generated by both users can later be used with a symmetric encryption algorithm such as AES or Blowfish to securely encrypt and decrypt messages.

The main objective of Diffie–Hellman is to enable secure key exchange even when attackers can observe all communications on the network.

Working of Diffie–Hellman Key Exchange

Assume two users, **Alice** and **Bob**, want to communicate securely.

Step 1: Public Parameters

Alice and Bob first agree on two public values.

Prime Number (P) = 23

Generator (G) = 5

These values are public and can be known by everyone, including attackers.

Step 2: Private Keys

Each user selects a private key that is kept secret.

Alice chooses private key = 6

Bob chooses private key = 15

These private keys are never shared with anyone.

Step 3: Public Key Generation

Each user generates a public key using the formula:

Public Key = $G^{\text{Private Key}} \bmod P$

Alice calculates:

$$A = 5^6 \bmod 23$$

$$A = 8$$

Bob calculates:

$$B = 5^{15} \bmod 23$$

$$B = 19$$

Alice sends **8** to Bob.

Bob sends **19** to Alice.

Even if an attacker captures these public keys, the private keys remain secret.

Step 4: Shared Secret Generation

Alice calculates the shared secret using Bob's public key.

$$\text{Shared Secret} = 19^6 \bmod 23$$

$$\text{Shared Secret} = 2$$

Bob calculates the shared secret using Alice's public key.

$$\text{Shared Secret} = 8^{15} \bmod 23$$

$$\text{Shared Secret} = 2$$

Both users independently obtain the same secret key **2** without ever transmitting it over the network.

This shared secret can now be used as the encryption key for secure communication.

Diagram of Diffie–Hellman Key Exchange

Public Values

$$P = 23, G = 5$$

Alice

Bob

Private = 6

Private = 15

|

|

Generate Public Key

Generate Public Key

$$A = 8$$

$$B = 19$$

| -----8-----> |

$$| \leftarrow 19 \rightarrow |$$

Compute Shared Key Compute Shared Key

$$19^6 \bmod 23 = 2 \qquad 8^{15} \bmod 23 = 2$$

Shared Secret Key = 2

The important feature of Diffie–Hellman is that the secret key is **never transmitted** across the public network. Only public values are exchanged, making the communication much more secure than directly sending the encryption key.

B. Identify a possible security threat and suggest a solution to mitigate it.

Although Diffie–Hellman securely establishes a shared secret key, it does not verify the identity of the communicating users. Because of this limitation, it is vulnerable to a **Man-in-the-Middle (MITM) attack**.

Man-in-the-Middle Attack

In this attack, an attacker secretly positions themselves between Alice and Bob. Instead of Alice communicating directly with Bob, the attacker intercepts all messages and creates separate secret keys with each user.

The attacker can then decrypt, read, modify, or even replace the messages before forwarding them. Alice and Bob believe they are communicating securely with each other, but in reality, the attacker controls the communication.

For example:

Alice sends her public key to Bob.

The attacker intercepts it and replaces it with their own public key.

Bob unknowingly creates a shared key with the attacker.

Similarly, the attacker creates another shared key with Alice.

As a result:

Alice ↔ Attacker ↔ Bob

Both Alice and Bob think they are communicating securely, while the attacker can read all exchanged information.

Solution to Mitigate the Attack

To prevent Man-in-the-Middle attacks, Diffie–Hellman should be combined with **authentication mechanisms**.

The following security measures can be used:

Use **Digital Certificates** issued by a trusted Certificate Authority (CA) to verify the identity of users.

Use **RSA Digital Signatures** to authenticate the exchanged public keys.

Use authenticated versions of Diffie–Hellman in secure protocols such as **TLS/SSL**, where certificates verify the communicating parties.

Verify public keys before generating the shared secret.

Use strong prime numbers and sufficiently large key sizes to improve overall security.

These techniques ensure that the exchanged public keys belong to the intended users and not to an attacker.

Advantages of Diffie–Hellman

Diffie–Hellman allows secure key exchange over an unsecured network.

The secret key is never transmitted.

It provides strong protection against eavesdropping.

It is widely used in VPNs, HTTPS, SSH, and secure communication protocols.

It supports secure communication between previously unknown users.

Limitations of Diffie–Hellman

It does not authenticate users.

It is vulnerable to Man-in-the-Middle attacks if authentication is not used.

It only establishes a shared secret and does not encrypt data by itself.

Additional encryption algorithms are required for secure message transmission.

Conclusion

Diffie–Hellman Key Exchange enables two users to establish a common secret key over an insecure network without transmitting the key directly. This makes it an effective method for secure key exchange. However, because it does not authenticate the communicating users, it is vulnerable to **Man-in-the-Middle attacks**. This weakness can be mitigated by combining Diffie–Hellman with **digital certificates, RSA digital signatures, and authenticated protocols such as TLS/SSL**. As a result, Diffie–Hellman provides secure and reliable key establishment for modern communication systems when used with proper authentication mechanisms.

Question 4. Public Key Cryptography for Email Security

Scenario

An organization plans to implement **RSA (Rivest–Shamir–Adleman)** public key cryptography to provide secure email communication between employees and external users. The organization wants to ensure that emails remain

confidential during transmission and that only the intended recipient can read them. It also requires proper key distribution and management to maintain long-term security.

A. Describe how RSA can be used for encryption and decryption in this scenario.

RSA is an **asymmetric encryption algorithm** that uses two different keys:

Public Key – Used for encryption and can be shared openly.

Private Key – Used for decryption and must be kept secret by the owner.

Unlike symmetric encryption, RSA does not use the same key for both encryption and decryption. This makes RSA highly suitable for secure email communication because users can freely distribute their public keys while protecting their private keys.

Working of RSA in Email Security

Suppose **Alice** wants to send a confidential email to **Bob**.

Step 1: Key Generation

Bob generates two mathematically related keys:

Public Key

Private Key

Bob shares only his **Public Key** with Alice, while keeping the **Private Key** secret.

Step 2: Email Encryption

Alice writes the email message.

Example:

"Transfer ₹5,00,000 to Account No. XXXXX."

Before sending the email, Alice encrypts the message using **Bob's Public Key**.

The plaintext email becomes encrypted ciphertext.

Ciphertext = RSA Encryption (Plaintext, Bob's Public Key)

The encrypted email is then transmitted over the Internet.

Even if an attacker intercepts the email, the contents cannot be understood because they are encrypted.

Step 3: Email Decryption

When Bob receives the encrypted email, he uses **his Private Key** to decrypt it.

Plaintext = RSA Decryption (Ciphertext, Bob's Private Key)

Since only Bob possesses the Private Key, only he can successfully recover the original email message.

This ensures that confidential information remains protected throughout transmission.

Step 4: Digital Signature (Authentication)

RSA can also provide **authentication and integrity** using digital signatures.

Before sending the email, Alice creates a hash value of the message using a hashing algorithm such as SHA-256.

She encrypts this hash using **her Private Key** to create a **Digital Signature**.

The digital signature is attached to the email.

When Bob receives the email:

He decrypts the signature using **Alice's Public Key**.

He computes a new hash of the received email.

If both hash values are identical, Bob knows that:

The email was sent by Alice.

The message has not been modified during transmission.

This protects the email from tampering and impersonation.

RSA Email Communication Process

Alice Writes Email

|

Encrypt using Bob's Public Key

|

Encrypted Email

|

Sent through Internet

|

Bob Receives Email

|

Decrypt using Bob's Private Key

|

Original Email Displayed

Benefits of RSA in Email Security

Provides confidentiality by encrypting emails.

Only the intended recipient can decrypt the message.

Supports digital signatures for authentication.

Ensures message integrity.

Protects against unauthorized access.

Widely used in secure email systems and online communication.

B. Explain how key distribution and management play a role in maintaining security.

The security of RSA depends not only on the encryption algorithm but also on proper **key distribution** and **key management**. Even the strongest encryption becomes ineffective if keys are distributed or managed improperly.

Key Distribution

Key distribution is the process of safely providing public keys to users.

Each user publishes their Public Key while keeping their Private Key secret.

To ensure that a public key truly belongs to its owner, organizations use **Digital Certificates** issued by a **Certificate Authority (CA)**.

A Digital Certificate contains:

User's identity

Public Key

Certificate validity period

Digital signature of the Certificate Authority

When Alice receives Bob's certificate, she verifies it before using Bob's Public Key. This prevents attackers from replacing Bob's Public Key with a fake one.

Key Management

Key management refers to the complete lifecycle of cryptographic keys.

It includes:

Generating strong RSA key pairs.

Securely storing private keys.

Distributing public keys safely.

Backing up keys securely.

Rotating keys periodically.

Revoking compromised or expired keys.

Replacing old keys with new ones.

Proper key management ensures that only authorized users have access to private keys and that compromised keys cannot continue to be used.

Importance of Key Distribution and Management

Ensures that public keys belong to genuine users.

Prevents Man-in-the-Middle attacks.

Protects private keys from theft.

Maintains confidentiality and authenticity.

Supports secure communication between employees.

Allows secure renewal and revocation of keys when required.

Improves trust in organizational email systems.

Advantages of RSA for Email Security

Provides strong encryption.

Supports secure key exchange.

Enables digital signatures.

Ensures confidentiality, integrity, authentication, and non-repudiation.

Widely accepted for secure email communication.

Easy to integrate with SSL/TLS and secure email protocols.

Limitations of RSA

Encryption is slower than symmetric algorithms.

Requires larger key sizes for higher security.

Needs proper certificate management.

Private key compromise can expose confidential emails.

Often combined with symmetric algorithms (such as AES) for encrypting large email content efficiently.

Justification

The organization requires a secure method to protect confidential emails from unauthorized access. RSA fulfills this requirement by using a **public key for**

encryption and a **private key for decryption**, ensuring that only the intended recipient can read the message. In addition, RSA digital signatures verify the sender's identity and detect any modification to the email. Proper key distribution through digital certificates and effective key management further strengthen the overall security of the email communication system.

Conclusion

RSA is an effective public-key cryptographic algorithm for secure email communication. It ensures **confidentiality** by encrypting messages with the recipient's public key and allows only the recipient to decrypt them using the private key. RSA also provides **authentication, integrity, and non-repudiation** through digital signatures. Proper **key distribution** using digital certificates and efficient **key management** are essential for maintaining trust, preventing attacks, and ensuring the long-term security of organizational email systems.

Question 5. Cryptography for IoT Devices

Scenario

A smart home system uses **low-power Internet of Things (IoT) devices** such as smart locks, smart lights, security cameras, motion sensors, smart thermostats, and wearable devices. These devices communicate over the Internet and exchange sensitive information. The system designer must choose between **RSA** and **Elliptic Curve Cryptography (ECC)** to provide secure communication while considering the limitations of IoT devices.

A. Analyze the constraints of IoT devices.

Internet of Things (IoT) devices are designed to perform specific tasks with limited hardware resources. Unlike computers and servers, IoT devices have restrictions in processing power, memory, battery life, and storage capacity. Therefore, lightweight cryptographic algorithms are required to ensure secure communication without consuming excessive resources.

Major Constraints of IoT Devices

1. Limited Processing Power

IoT devices generally use low-speed microcontrollers instead of powerful processors. Complex encryption algorithms require more computational time, increasing processing delay and reducing system performance.

Example:

A smart temperature sensor cannot spend several seconds performing encryption because it must continuously monitor and send temperature readings.

2. Limited Memory

Most IoT devices have very small RAM and storage space.

Large cryptographic algorithms require more memory for key storage and mathematical calculations. This makes it difficult for resource-constrained devices to execute complex security operations.

3. Limited Battery Power

Many IoT devices operate using batteries for months or even years.

Performing computationally intensive encryption repeatedly consumes more power and reduces battery life.

For example:

A wireless door sensor should continue operating for a long time without frequent battery replacement.

4. Limited Storage Capacity

IoT devices cannot store large cryptographic keys, certificates, or security software due to limited internal storage.

Efficient algorithms that require smaller key sizes are therefore preferred.

5. Real-Time Communication

Smart home devices must communicate quickly.

For example:

Motion sensors should instantly notify homeowners when movement is detected.

Heavy encryption algorithms may increase communication delay.

6. Large Number of Connected Devices

A smart home may contain dozens or even hundreds of IoT devices.

The cryptographic algorithm must support secure communication among multiple devices without causing excessive computational overhead.

7. Security Threats

Since IoT devices are connected to the Internet, they face various cyber threats such as:

Unauthorized access

Data interception

Device impersonation

Man-in-the-Middle attacks

Malware attacks

Replay attacks

Therefore, the selected encryption algorithm must provide strong security while remaining lightweight.

Summary of IoT Constraints

Low processing capability.

Limited RAM and storage.

Limited battery life.

Need for fast communication.

Large number of connected devices.

High security requirements.

Low computational overhead.

B. Recommend a suitable cryptographic approach and justify your answer.

Among the two available options, **Elliptic Curve Cryptography (ECC)** is the most suitable cryptographic approach for IoT devices.

ECC is an asymmetric encryption algorithm that provides the same level of security as RSA while using much smaller key sizes. Smaller keys reduce computation time, memory usage, storage requirements, and power consumption, making ECC ideal for resource-constrained IoT devices.

For example:

A **256-bit ECC key** provides security comparable to a **3072-bit RSA key**.

Because of its smaller key size, ECC performs encryption, decryption, and digital signature operations much faster while consuming significantly fewer hardware resources.

Why RSA is Less Suitable

RSA provides strong security but has several limitations when used in IoT devices.

RSA requires large key sizes such as 2048 or 3072 bits.

Larger keys increase CPU usage.

Encryption and decryption require more processing time.

Power consumption is higher.

More memory and storage are required.

These characteristics make RSA less suitable for low-power embedded devices.

Advantages of ECC

Uses much smaller key sizes.

Consumes less battery power.

Requires less memory.

Faster encryption and decryption.

Lower computational complexity.

Provides strong security.

Suitable for wireless communication.

Ideal for smart home devices and embedded systems.

Supports digital signatures and secure key exchange.

Comparison Between RSA and ECC

Feature	RSA	ECC
Key Size	Large (2048–3072 bits)	Small (160–521 bits)
Security	High	High
Processing Speed	Slower	Faster

Feature	RSA	ECC
Memory Requirement	High	Low
Power Consumption	High	Low
Storage Requirement	High	Low
Suitable for IoT	Less Suitable	Highly Suitable

Justification

The smart home system consists of battery-powered devices with limited processing power, memory, and storage. These devices require fast and secure communication without consuming excessive energy. ECC satisfies all these requirements because it provides strong encryption using much smaller keys than RSA. As a result, ECC reduces computation time, minimizes battery consumption, saves memory, and improves communication efficiency while maintaining a high level of security.

Therefore, ECC is the best choice for protecting communication between IoT devices.

Real-Life Applications of ECC

- ECC is widely used in:
 - Smart home automation systems.
 - Smart watches and wearable devices.
 - Wireless sensor networks.
 - Mobile banking applications.
 - Internet of Things (IoT) devices.
 - Bluetooth communication.
 - Secure messaging applications.
 - Digital certificates.
 - SSL/TLS secure communication.
-

Conclusion

IoT devices have several limitations, including **limited processing power, memory, storage, battery life, and the need for real-time communication**. These constraints make lightweight cryptographic algorithms essential. Compared with RSA, **Elliptic Curve Cryptography (ECC)** offers the same level of security while using much smaller keys, resulting in faster processing, lower power consumption, and reduced memory usage. Therefore, **ECC is the most suitable cryptographic approach for securing communication in smart home IoT systems**, providing efficient, reliable, and scalable security for resource-constrained devices.